

04. From Text to Attention — Part 2

What is Self-Attention? (The Big Idea)

- Every word in a sentence looks at **every other word** to understand its own meaning better
- This happens **all at once** — not one word at a time
- The result is that each word gets a **context-aware meaning** — it now knows what the other words mean for it
- This is the most important part of the whole transformer

Think about this sentence: *"The animal didn't cross the street because it was too tired."*

What does "it" mean? You instantly know it means "the animal" — not "the street." You figured that out by looking at the other words. Self-attention does exactly the same thing, but with maths.

Real-life analogy: Imagine you're in a group project. Before you start your part, you talk to every other member to understand what they're doing. That conversation changes how you understand your own role. Self-attention is that conversation — every word "talks" to every other word before deciding what it really means.

Key takeaway: Self-attention lets every word update its meaning based on every other word in the sentence — all at the same time.

Query, Key, and Value — The Three Roles of Every Word

This is where most people get confused. Let's go slow.

Every word plays **three roles at once** when doing attention. These roles are called **Q (Query), K (Key), and V (Value)**.

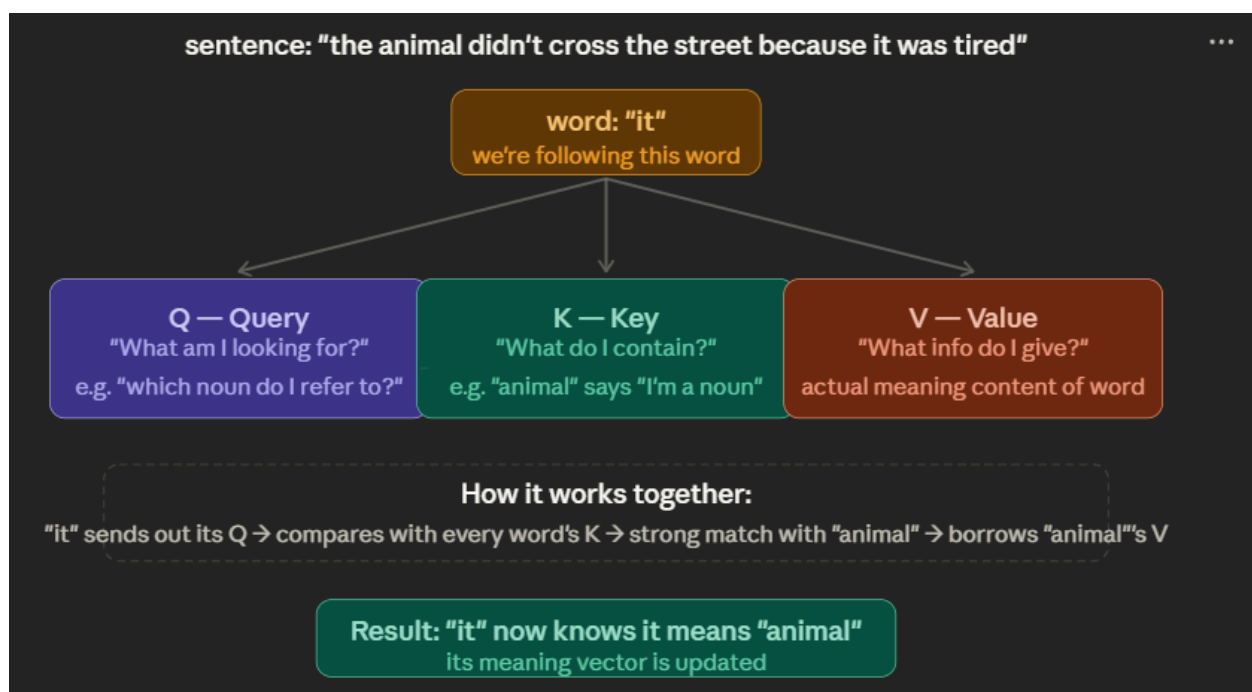
Here's the simplest way to understand each one:

Real-life analogy — Think of a job fair:

- You walk in with a **Query** — "I'm looking for a software engineering job"
- Every company has a **Key** on their booth — "We hire software engineers", "We hire designers", etc.
- If your Query matches their Key, you go talk to them — and they hand you their **Value** — the actual information: salary, role details, perks.

So in a sentence:

- **Q** = "What am I looking for?" — each word asks a question about the rest of the sentence
- **K** = "What do I contain?" — each word advertises what it's about
- **V** = "What do I actually give you?" — the real content passed on when there's a match



So how do we actually get Q, K, and V? Each word's embedding (its meaning vector from Part 1) is multiplied by three separate **weight matrices** — think of

them as three different "lenses" through which the model looks at each word. These matrices are learned during training, so the model figures out the best way to create Q, K, V on its own.

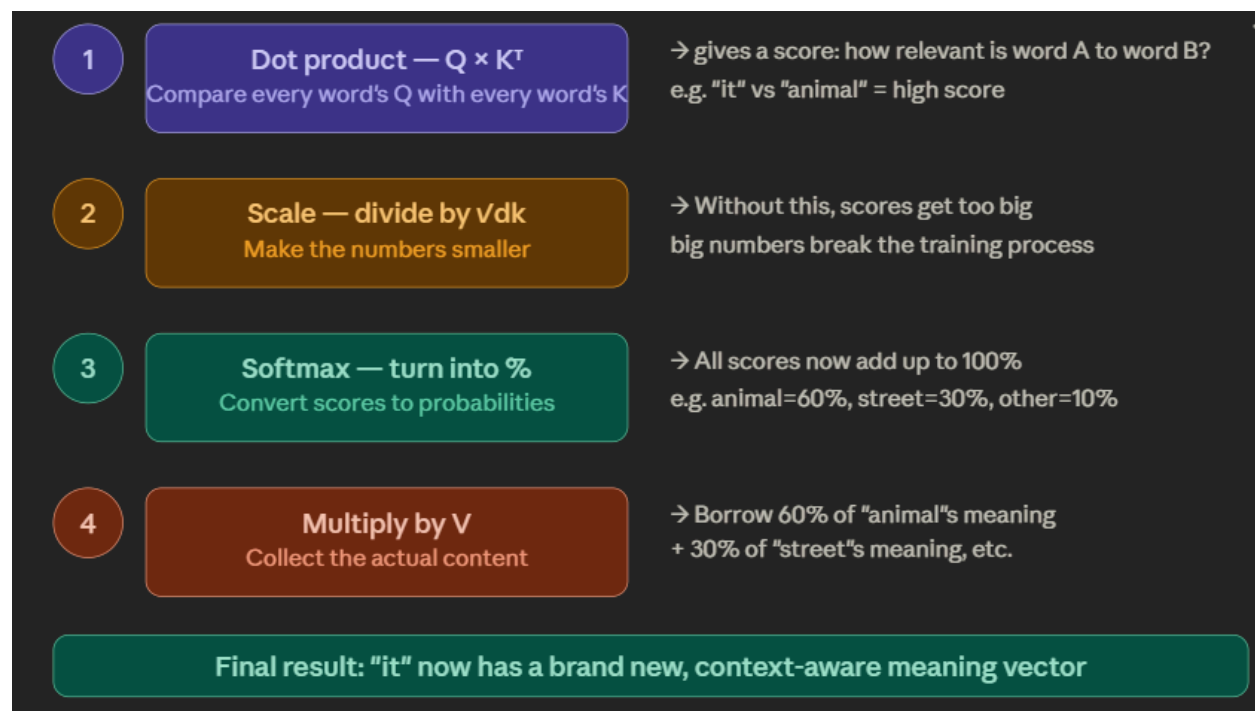
Key takeaway: Every word asks a question (Q), wears a label (K), and carries real content (V). Attention is just the process of matching questions to labels and then collecting the content.

The Attention Formula — Plain English Walkthrough

The formula looks scary. Let's break it into 4 tiny steps.

$$\text{Attention}(Q, K, V) = \text{Softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) \times V$$

Don't panic — here's what each part actually does:



The most important thing to understand about Step 3 (Softmax) is this: attention is **never all-or-nothing**. It's always a mix. The word "it" borrows a little bit from every word — just more from "animal" and less from everything else. That's why it's called a "soft" lookup, not a "hard" one.

Simple example — Hard vs Soft lookup:

Hard (like a dictionary): you type "cat" and get exactly one result.

Soft (like attention): you type "it" and get 60% animal, 30% street, 10% everything else — all blended together.

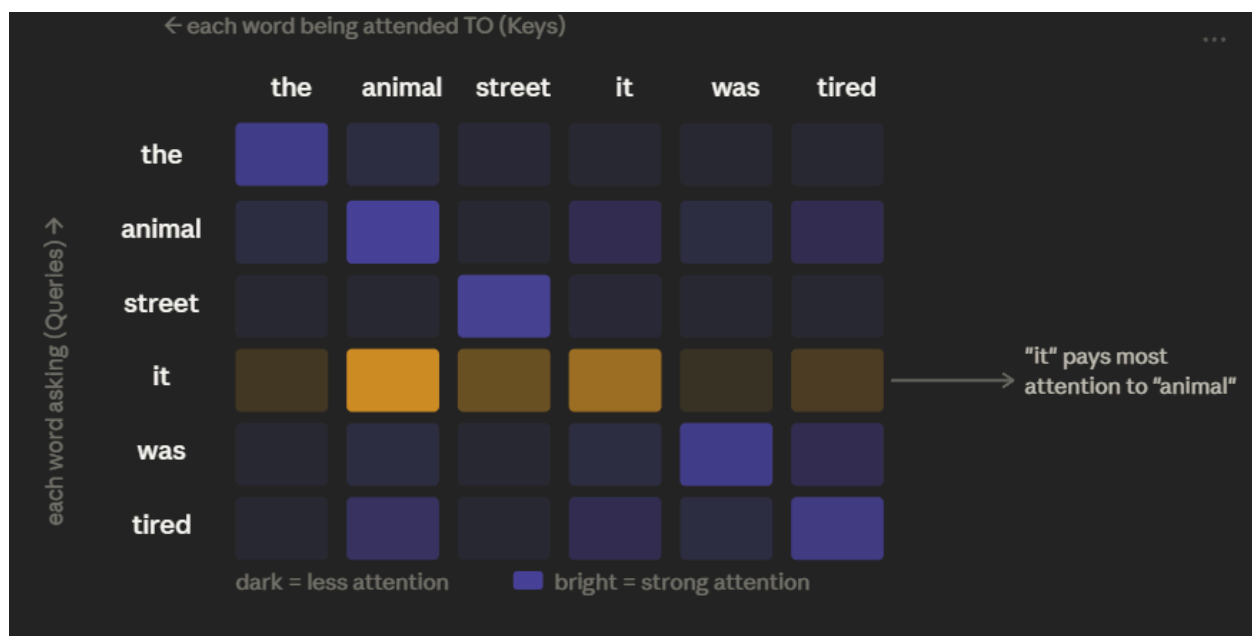
Key takeaway: The attention formula is just 4 steps: score how relevant each word is → shrink the numbers → convert to percentages → mix in the real content.

The Attention Matrix — Seeing the Connections

After doing attention for every single word in a sentence, you end up with a grid. This grid is called the **attention matrix**.

- Rows = each word asking a question
- Columns = each word being asked
- Each cell = how much attention word A pays to word B
- **Bright cell = strong connection. Dark cell = weak connection.**

Think of it like a spreadsheet where every word rates every other word's relevance to itself — from 0% (ignore this word) to high% (pay a lot of attention to this word).



Look at the row for "it" (shown in orange). The brightest cell is in the "animal" column. That means "it" is paying the most attention to "animal." The model has

correctly figured out what "it" refers to — just from the maths of Q, K, and V.

Key takeaway: The attention matrix is a map of how much every word cares about every other word. Bright = strong connection.

KV Cache — Why the First Word is Slow and the Rest are Fast

This is a clever speed trick. First, let's understand the problem it solves.

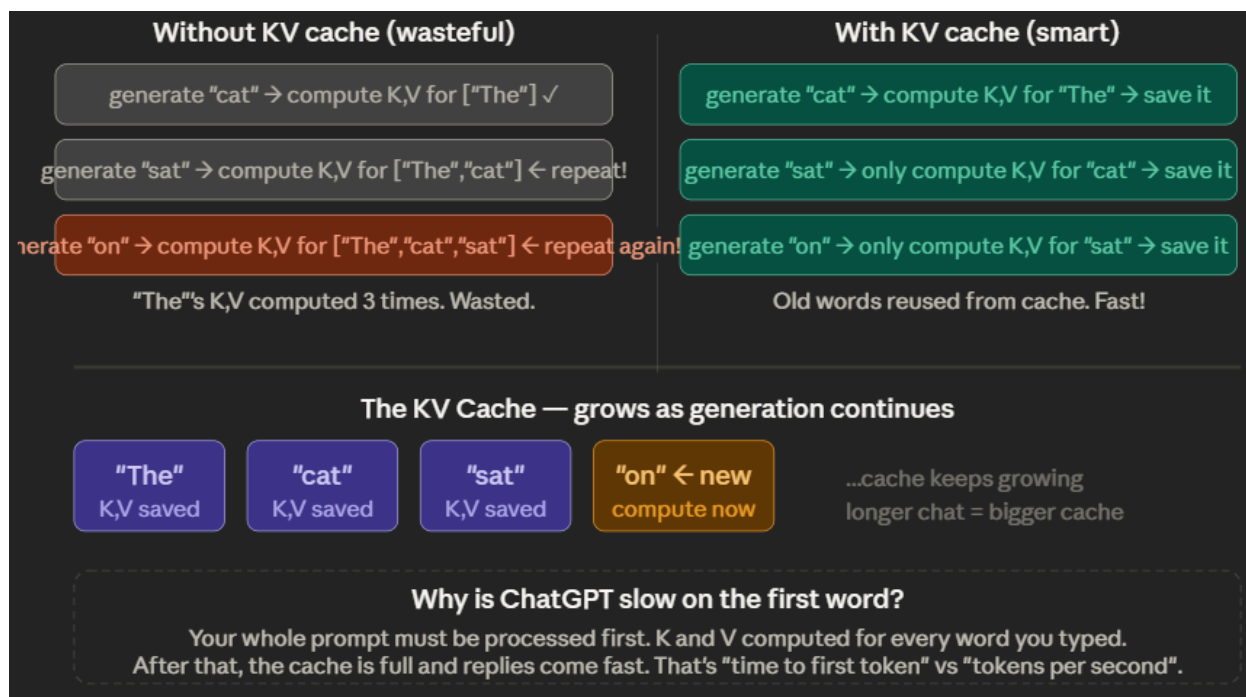
When a model like ChatGPT generates a reply, it creates the response **one word at a time**. To generate each new word, it runs the full attention calculation. The problem? It was doing a lot of **repeated, wasted work**.

Simple example — Generating "The cat sat on":

- To generate "cat" → compute attention for ["The"]
- To generate "sat" → compute attention for ["The", "cat"]
- To generate "on" → compute attention for ["The", "cat", "sat"]

Notice: "The"'s K and V were computed 3 times. "cat"'s K and V were computed twice. That's pure waste.

The insight is simple: **K and V for old words never change**. Only the new word needs fresh K and V. So why not just save them?



There's one important trade-off though. The cache gets **bigger** the longer your conversation gets. A short chat = small cache = fast. A very long document = huge cache = needs a lot of memory. This is the real reason why long context windows are hard to build — storing the cache for 100,000 tokens takes enormous memory, especially since it's stored separately for every layer of the model.

Key takeaway: The KV Cache is a simple memory trick — save the old work, only compute the new word. This is why generation gets faster after the first word.

Multi-Head Attention — Looking From Many Angles at Once

So far, we've done attention once — with one set of Q, K, V. But there's a problem. Language has **many different kinds of relationships** between words — and one attention pass can only see one type at a time.

Real-life analogy — Reading a news article:

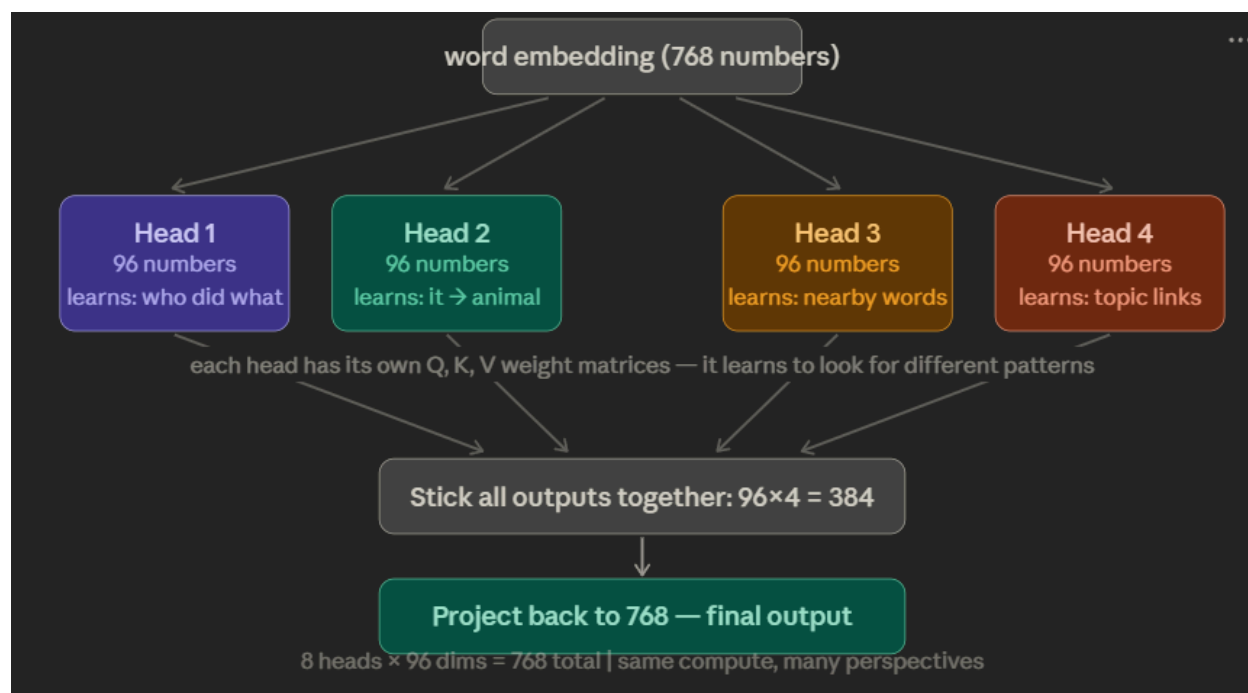
One reader might focus on *who did what* (subject-verb).

Another might focus on *when and where* (time and place).

Another might focus on *which words repeat* (topic links).

They all read the same article — but they each spot different things.

Multi-Head Attention is like running all three readers at the same time.



The clever bit is that the total amount of computation stays the same. Instead of one head using all 768 numbers, you split them into 8 heads of 96 numbers each. Same total — but now you get 8 different perspectives. After each head does its own attention, all the outputs are glued back together and projected back to 768 numbers.

Why not use 100 heads then? Because each head gets smaller and smaller — too many heads means each one only sees a tiny sliver and can't learn anything useful. In practice, 8–32 heads is the sweet spot.

Key takeaway: Multi-head attention is like reading the same sentence with multiple pairs of eyes simultaneously — each pair spots a different kind of relationship, and then all the insights are combined.

Master Summary Table

Topic	What it is, in plain English	Why it matters
Self-attention	Every word looks at every other word to understand its own	Words have different meanings in different contexts — this is how the

Topic	What it is, in plain English	Why it matters
	meaning better	model figures that out
Q (Query)	The question a word is asking — "what am I looking for?"	Drives the search process
K (Key)	The label a word wears — "what do I contain?"	Gets matched against the Query
V (Value)	The actual meaning a word shares when matched (what can i offer)	This is what gets mixed into the output
Attention formula	Score → shrink → convert to % → mix in content	Turns Q, K, V into a new context-aware meaning for each word
Attention matrix	A grid showing how much each word pays attention to every other word	Makes attention visual and interpretable
Soft vs hard lookup	Hard = exact match only. Soft = blended mix of partial matches	Soft lookup is more flexible and handles ambiguity
KV Cache	Save old words' K and V so you don't recompute them every step	Makes text generation much faster after the first word
Why first token is slow	Your whole prompt must be processed and cached before generation starts	Explains "time to first token" vs "tokens per second"
Multi-head attention	Run attention 8 times in parallel, each looking for different patterns	One pass can't capture all relationship types in language
Head count trade-off	More heads = more perspectives, but each gets a smaller dimension	Too many heads = each too small to learn anything useful